

S-Curve with pre-braking

Miya Su

April 10, 2026

Abstract

S-curve trajectory generation is executed during system state transitions, such as active disengagement or deceleration to a stop state. During operation, failure cases were encountered where the algorithm became numerically unstable. Investigation through recorded data replay in simulation revealed that instability occurred when the initial kinematic state — specifically the initial acceleration — approached a critical value that causes the trajectory duration equation to become singular. To address this, a pre-braking phase was developed that brings the system's kinematic state back within safe bounds before the main S-curve is executed, eliminating the instability across all tested operating conditions. The pre-braking phase is a modified version of the Ruckig algorithm, adapted to the specific kinematic constraints of the system.

Contents

1	Implementation Summary	3
1.1	Algorithm Analysis	3
1.2	Pre-Braking Phase	3
2	Modified Ruckig Pre-Braking Profile	4
2.1	High-Level Logic & Calling Structure	4
2.2	Implementation Strategies (The "Braking" Cases)	5
3	Summary	5

1 Implementation Summary

1.1 Algorithm Analysis

Customers reported invalid trajectory computations when the initial kinematic state falls outside the expected bounds. In the legacy S-curve implementation, the duration of the whole trajectory, T_{min} , is pre-computed. It was defined by the following expression:

$$T_{min} = \frac{1}{6a_{max}(2a_{max}(n-1) + a_0)^2v_{max}} \left[\begin{aligned} &- 24a_{max}^3(n-1)^2(p_0 - p_e) \\ &+ 12a_{max}^2(n-1)(n(v_0^2 - 2v_0v_{max} + 2v_{max}^2) - 2a_0(p_0 - p_e)) \\ &- 2a_0a_{max}(3a_0(p_0 - p_e) - 2(v_0^2 - 2v_0v_{max} + (3n+1)v_{max}^2)) \\ &+ 3a_0^2v_{max}^2\frac{n}{n-1} \end{aligned} \right] \quad (1)$$

Mathematically, the validity of this equation requires a non-zero denominator. A singularity occurs when:

$$a_0 = -2a_{max}(n-1) \quad (2)$$

Using the parameters $v_{max} = 0.1$, $a_{max} = 0.2$, and $n = 6$, the critical value is $a_0 = -2(0.2)(5) = -2.0$. As illustrated in Figure 1, as a_0 approaches this value, T_{min} tends toward infinity, leading to numerical instability.

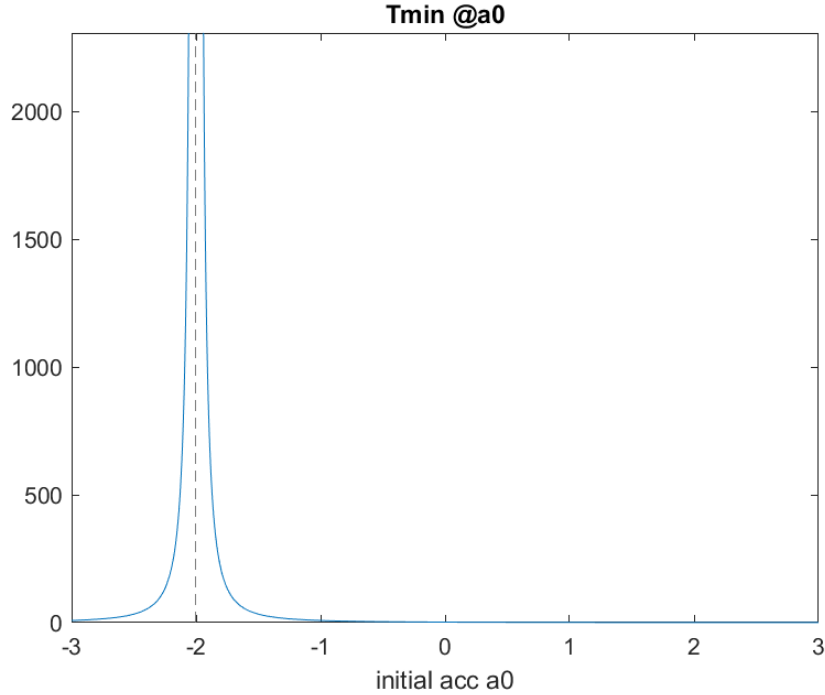


Figure 1: T_{min} dependency on initial acceleration a_0 .

1.2 Pre-Braking Phase

Analysis shows that the location of the mathematical singularity is not static; rather, it varies as a function of the kinematic constraints v_{max} , a_{max} , and the boundary positions p_0 and p_e .

Because this instability shifts depending on the motion profile, a hard-coded safety margin is insufficient. This necessitates a robust architectural change: the integration of a **Pre-Braking Phase** inspired by the Ruckig algorithm. Unlike the legacy approach, which attempts to solve the entire motion with a single unstable equation, the pre-braking phase decouples the problem into two distinct steps:

1. **State Correction:** Preliminary maneuvers bring the system’s velocity or acceleration back within defined limits.
2. **Main Trajectory:** Once the state is guaranteed to be within a safe, non-singular region, the main S-curve is generated.

2 Modified Ruckig Pre-Braking Profile

The primary purpose of the pre-braking phase is to calculate ”braking” trajectories—preliminary maneuvers used to bring a system’s state (velocity or acceleration) back within its defined limits before the main trajectory begins. The algorithm uses a conditional branching structure to identify the ”worst-offender” kinematic limit. By swapping arguments and flipping the sign of j_{Max} , we use the same core logic to handle both positive and negative boundary violations.

2.1 High-Level Logic & Calling Structure

```
if (a0 > aMax)
    [t0,t1,t2...] = acceleration_brake(v0, a0, vMax, vMin, aMax, aMin, jMax);
elseif (a0 < aMin)
    [t0,t1,t2...] = acceleration_brake(v0, a0, vMin, vMax, aMin, aMax, -jMax);
elseif (v0 > vMax && v_at_aiszero(...) > vMin) || (a0 > 0 && v_at_aiszero(...) > vMax)
    [t0,t1,t2...] = velocity_brake(v0, a0, vMax, vMin, aMax, aMin, jMax);
elseif (v0 < vMin && v_at_aiszero(...) < vMax) || (a0 < 0 && v_at_aiszero(...) < vMin)
    [t0,t1,t2...] = velocity_brake(v0, a0, vMin, vMax, aMin, aMax, -jMax);
```

acceleration_brake

This function is triggered when the initial acceleration a_0 is outside the allowed bounds $[a_{min}, a_{max}]$.

- It calculates the duration required to bring the acceleration back to a_{max} or a_{min} .
- It performs a predictive check using `v_at_a_zero` to determine if ramping to zero acceleration will cause a velocity limit violation.
- If an overshoot is likely, it implements a Fast Recovery 3-segment profile (flipping a_0 to $-a_0$) or hands control to the `velocity_brake` logic.
- Otherwise, it computes the simple duration needed to reach the acceleration limit.

velocity_brake

This is utilized when the velocity v_0 exceeds its limits or is on a trajectory that will inevitably lead to a violation.

- It calculates the time needed to apply maximum jerk to reduce velocity as quickly as possible.
- It accounts for two distinct phases: a **ramping phase** (varying acceleration) and a **constant acceleration phase** (coasting at a_{min} or a_{max}) if the acceleration limit is reached before the velocity target is met.

- It includes a **1-segment** shortcut if the velocity limit is reached before the acceleration ramp-down can be completed.

2.2 Implementation Strategies (The "Braking" Cases)

The algorithm selects one of three primary jerk-limited profiles based on the severity of the state:

Strategy	Jerk Sequence	Description
Fast Recovery (3-Seg)	$[-j_{max}, 0, j_{max}]$	The "Symmetric Flip": Flips $a_0 \rightarrow -a_0$ to cancel velocity.
Velocity Coast (2-Seg)	$[-j_{max}, 0]$	Ramp & Constant: Ramps to a_{limit} and holds until v_{target} .
Immediate Exit (1-Seg)	$[-j_{max}]$	Direct Ramp: Used when v_{limit} is hit during the ramp.

3 Summary

The transition to a **S-Curve with pre-braking** architecture addresses a critical flaw in the legacy S-curve implementation, where the pre-computed trajectory duration T_{min} suffered from a mathematical singularity. When the initial acceleration a_0 reached the critical value $a_0 = -2a_{max}(n - 1)$, the denominator of the duration equation approached zero, causing T_{min} to tend toward infinity and inducing numerical instability.

By integrating a **Pre-Braking Phase** inspired by the Ruckig algorithm, the motion is now decoupled into two distinct stages: **State Correction** and **Main Trajectory**. The preliminary maneuvers utilize a conditional branching structure to identify kinematic "worst-offenders" and apply specific jerk-limited strategies: **Fast Recovery (3-Seg)**, **Velocity Coast (2-Seg)**, or **Immediate Exit (1-Seg)**. This architectural shift ensures that the system's velocity and acceleration are brought within safe limits before the main S-curve begins, effectively bypassing the regions of mathematical instability.